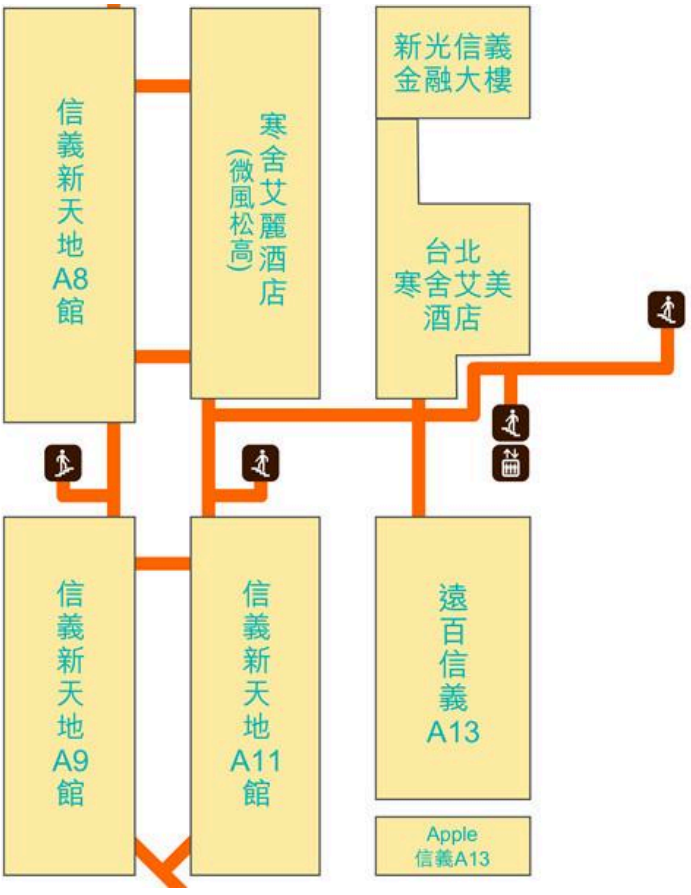
課程三 - AWS EC2 VPC Peering

1-1：VPC Peering

1-1-2：空橋系統：在先前的章節裡面，我們提到了雲端的網路VPC架構，可以建構屬於自有的雲端網路環境。但如果有多個VPC之間，如果要達到網路互相溝通，這時候就需要將兩個不同的VPC對等互連，也就是所謂的Peering。我們拿一個例子來說明，在繁華的信義商圈，有各種不同的百貨、娛樂等匯集，大樓林立。為了讓消費者可以省去穿越馬路，還可以遮陽擋雨，所以建立了空橋，最後便利的系統就產生了：

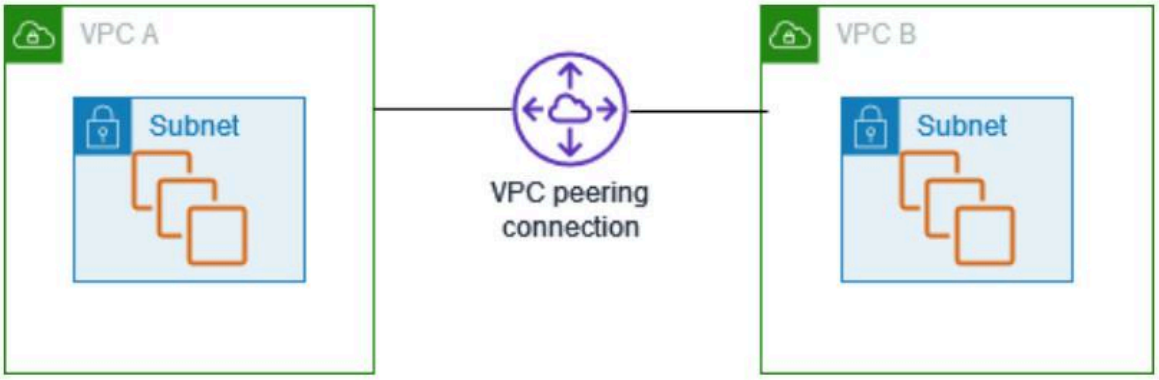


圖片來源：維基百科（信義商圈空橋系統）

1-1-2：Peering：那麼VPC Peering就是在兩個 VPC 之間建立私有網路連接，讓它們可以像在同一個網路中一樣互相通訊。如果不使用Peering，就必須先將流量傳送到網際網路，再透過網際網路連線到另一個VPC，但這樣就會有流量出(Data Transfer Out,DTO)的費用。

我們用簡單的示意圖來說明關於peering架構，我們不需要將流量傳送到網際網路，兩個VPC之間就形成了一個內網架構，可以讓流量封包傳遞，當然路由也是必須要隨之調整才可能串通。

VPC A CIDR = 10.0.0.0/16　　VPC B CIDR = 10.0.1.0/16



如果有第三個VPC，假設暫時稱為VPC C與VPC B互連，網路互通，那麼我們可以認定VPC A可以連線到VPC C嗎?答案是：不行。因為沒有支援所謂的路由傳遞（Non-Transitive）。然而如果VPC A連線到VPC C，我們則需要再建立VPC A與VPC C的互連。就會類似下面的概念。

VPC-A ←→ VPC-B
↓　　↓
└──────→ VPC-C

需要建立 3 個 Peering 連接:
- VPC-A ←→ VPC-B
- VPC-B ←→ VPC-C
- VPC-A ←→ VPC-C

1-1-3: 多帳號的網路環境串聯:

那麼這個概念很好，但如果越來越多的帳號且橫跨不同的區域 (Region)要互相連通，會有甚麼問題。你會遇到太多複雜的網路設定，所以這時候可以考慮Transit Gateway(Hub and Spoke架構)，不只可以跨帳號同時且支援路由傳遞，設定上就越便利了。不過需要額外付出一些成本才能滿足這個框架。

TGW-US (us-east-1)
  ├─ VPC-A
  └─ VPC-B
     │
     │ TGW Peering
     │
TGW-EU (eu-west-1)
  ├─ VPC-C

VPC-A 可以透過 TGW-US → TGW-EU 連到 VPC-C
只需 1 個 TGW Peering 連接

我們將這些上述做成表格就可以一目了然，在技術層面上的應用。

特性	**VPC Peering**	**Transit Gateway Peering**
連接對象	VPC ↔ VPC	TGW ↔ TGW
範圍	同 / 跨 Region	同 / 跨 Region
連接數量	1 對 1	多對多 (透過 TGW)
傳遞路由	不支援	支援
管理複雜度	高 (每對 VPC 需單獨設定)	低 (集中管理)
頻寬限制	無	50 Gbps (per TGW)
延遲	最低	稍高 (多一跳)
成本	低	高

以成本分析面的比較

需求	**VPC Peering**	**TGW Peering**
2 個 VPC 跨 Region	推薦	過度設計
5+ 個 VPC 跨 Region	管理困難	推薦
需要傳遞路由	不支援	支援
成本	相對便宜	相對較貴
多Region	複雜	推薦
低延遲需求	推薦	稍高

1-1-4：Shared VPC：但是網路分配得不好，又要知道怎麼分配，管理上一定會遇到問題，所以在企業、金融對於網路的管理相對重要，所以又產生另一個架構，名為共享VPC (Shared VPC)，然而這個架構僅存在於已經建立Organizations，同時將一個 VPC 的 Subnet 共享給其他 AWS 帳號使用，實現集中管理網路，分散管理應用

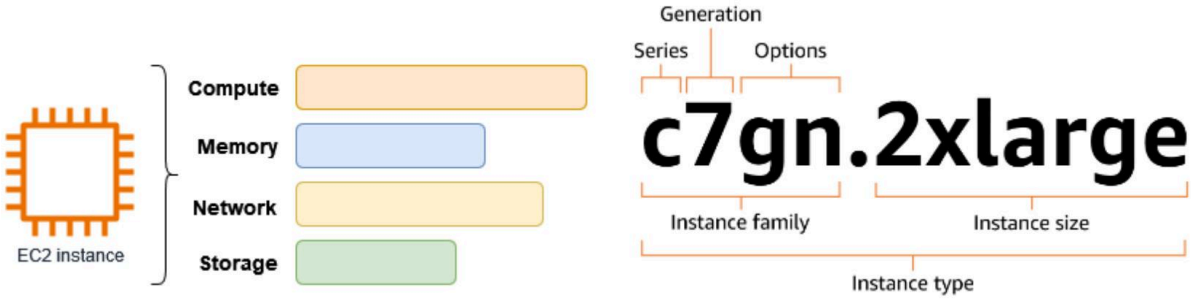
示意圖如下：

AWS Organization
│
├─ Management Account (111111111111)
│
├─ Network Account (222222222222) ← Owner
│ │
│ └─ VPC (10.0.0.0/16)
│   ├─ Public Subnet (10.0.1.0/24) ← 分享給 App Account
│   ├─ Private Subnet (10.0.2.0/24) ← 分享給 App Account
│   └─ DB Subnet (10.0.3.0/24)   ← 分享給 Data Account
│
├─ App Account (333333333333) ← Participant
│ └─ 在 10.0.1.0/24 部署：
│   ├─ EC2 (Web Server)
│   └─ ALB
│
└─ Data Account (444444444444) ← Participant
   └─ 在 10.0.3.0/24 部署：
      └─ RDS Database

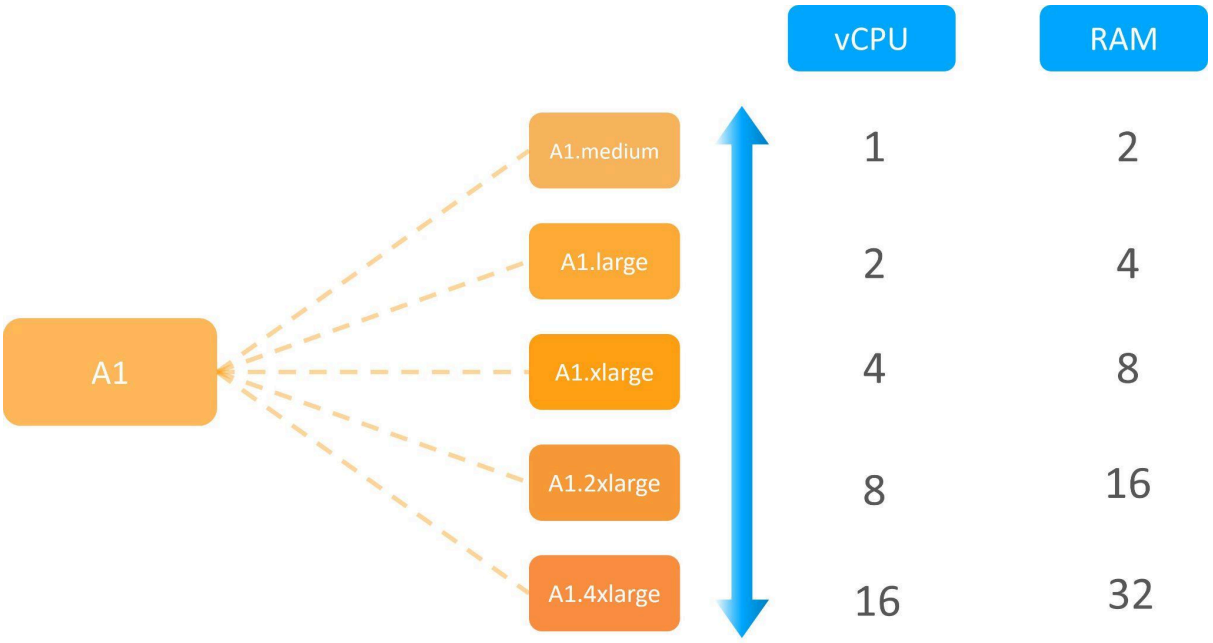
2-1：EC2 (Elastic Compute Cloud)

2-1-1：資源型態

我們搭建好網路環境，但還需要有一些運算的服務來搭建我們系統或是網站，而進入雲端後第一個服務就是EC2，我們可以當作這個服務是屬於雲端虛擬機，可以在雲端快速啟動和管理運算資源。想像一下我們使用電腦、筆電等組成有CPU、記憶體、硬碟、作業系統或是一些其他附屬的工具或應用程式。



可以彈性的使用對應的資源類型



如同在最佳實踐所提到的資源型態，M8i.xlarge，在雲端的規格也可以有以下內容選擇，然而作業系統的安裝則是使用雲端提供的作業系統建置，並且隨時保持最新可被調用的版本，並且使用預先配置好的作業系統映像檔來建置。但應該使用哪一種類型的運算資源呢?下方的表格簡單彙整參考，至於需要哪一種類型，可以參考官方的連結：
https://docs.aws.amazon.com/zh_tw/ec2/latest/instancetypes/instance-types.html

類型	常見場景	特色
加速運算	ML 模型訓練、圖形渲染	提供 GPU、AI 專用運算晶片
CPU 密集運算	ML inference、影片轉檔 (C Series)	提供較佳的 vCPU 性價比
記憶體高效	ERP 系統、快取資料庫 (M/R Series)	提供較高的 RAM 與 vCPU 比例
儲存高效 **I/O**	數據分析平台、資料倉儲	提供高速 SSD、HDD 的儲存空間
通用類型	Web 應用、API 服務 (C Series)	無特定需求下，性價比最佳
T Series	假設 vCPU 2 MEM 4	會有資源的使用限制

2-1-2：硬碟（EBS,Elastic Block Store）

硬碟可以獨立存在，並且多種不同的類型，比如SSD、HDD、IOPS等，或是當系統故障時，硬碟也支援移轉到其他的虛擬機進行掛載。然而之前提過的安全性，硬碟在選擇預設是沒有啟用加密的，必須手動啟用，但並非每個場景都需要使用加密技術(AES-256)，原先AWS在2006年推出的時候EBS的加密技術還不存在，直到了2013年左右才推出EBS加密功能，為了維持原先客戶的工作流程，這個功能預設不啟用。但當我們啟用了加密，即使使用快照硬碟，還是屬於加密的硬碟，這時候我們的作法是：

步驟 1: 建立加密磁碟區的快照

步驟 2: 啟動臨時 EC2實例

步驟 3: 從加密快照建立加密磁碟區

步驟 4: 掛載加密磁碟區到臨時實例

步驟 5: 建立未加密的空白磁碟區

步驟 6: 掛載未加密磁碟區

步驟 7: 在實例內複製資料

#步驟 8: 卸載兩個磁碟區

步驟 9: 將未加密磁碟區掛載到目標實例

2-1-3：Key Pair (金鑰)

用於安全連線到 EC2 實例，每個區域最多可建立 5,000個金鑰對，公鑰存在 AWS，私鑰由用戶保管。Linux 使用 SSH（Security Group Port 22 Allow），Windows 使用 RDP（Security Group Port 3389 Allow）

2-1-4：EC2扮演的腳色

EC2既然是一臺虛擬機，擁有了作業系統與其他的規格，所以可以拿來安裝網站系統、資料庫系統等等，因為EC2就是一臺虛擬機。

2-1-5:運算的成本

運算資源相對消耗較多的成本，所以在AWS上針對節費有一些方案:

計費模式	適用場景	費用	限制
On-Demand	工作負載不容中斷，需穩定持讀的運算資源	最高	無 (彈性最高)
Reserved Instance (RI)	工作負載不容中斷，需穩定持讀的運算資源，且可以確定長期(1年或3年)需要的機款	低	依合約使用指定機款
Spot Instances	工作負載可以中斷，系統為運算與儲存低耦和的任務 (例 ETL)	最低	運算資源隨時被收回 (提前2分鐘通知) 用戶需實作能容忍中斷的re-try 機制

3-1:實作環節（建立一個雲端網站）

建立雲端資源，下方的User Data可以作為匯入使用

```
#!/bin/bash
yum update -y
yum install -y httpd
systemctl start httpd
systemctl enable httpd

echo "<h1>Hello World</h1><p>歡迎來到 AWS EC2</p>" > /var/www/html/index.html

# 設定檔案權限
chmod 644 /var/www/html/index.html
```

3-2:實作環節（建立一個雲端VPN)

